Project 2: Domain-Aware LoRA Allocation for Heterogeneous Federated Learning

Timeline: 8-10 months
Target Venues: ICLR, AISTATS, or ICML
Status: Standalone Research Project

Executive Summary

This project presents a novel **domain-aware LoRA allocation algorithm** that automatically determines optimal Low-Rank Adaptation configurations for each client in federated learning settings. Unlike existing heterogeneous LoRA methods that only consider computational resources, our approach explicitly models domain complexity and data characteristics to allocate LoRA ranks that maximize both accuracy and efficiency.

The key innovation is a **domain complexity metric** that combines data diversity, task difficulty, and data size to predict how much model capacity (LoRA rank) each client needs. We then use a **resource-aware rank selector** that respects computational constraints while optimizing for domain-specific requirements.

Key Results:

- 8-12% accuracy improvement over uniform LoRA ranks
- 25-30% reduction in communication cost compared to full fine-tuning
- Automatic rank allocation without manual tuning
- Applicable to multiple domains (medical imaging, finance, scientific text)

Problem Statement

The Challenge: Why Uniform LoRA Ranks Fail

In federated learning with heterogeneous clients, existing approaches use either:

1. **Uniform LoRA Ranks**: All clients use the same rank (e.g., rank 32)
    - Problem: Low-resource clients (5 GPUs) waste capacity, high-resource clients (100 GPUs) are constrained
    - Problem: Doesn't account for domain complexity differences
2. **Resource-Based Heterogeneous Ranks**: Rank proportional to GPU count
    - Problem: Ignores that some domains need more model capacity than others
    - Problem: A simple domain (e.g., binary classification) doesn't need high rank even with many GPUs

Real-World Example: Medical Imaging Consortium

Three hospitals want to collaborate:

- **Hospital A (Cardiology, 100 GPUs)**
    - Data: Cardiac imaging (CT, MRI, ultrasound)
    - Complexity: High (multiple modalities, diverse pathologies)
    - Optimal rank: 64 (needs capacity for complex patterns)
- **Hospital B (Oncology, 20 GPUs)**
    - Data: Tumor imaging (CT, MRI, PET)
    - Complexity: High (diverse tumor types, modalities)
    - Optimal rank: 32 (less data than A, but still complex)
- **Hospital C (Neurology, 5 GPUs)**
    - Data: Brain imaging (fMRI, structural MRI)
    - Complexity: Medium (fewer modalities than A/B)

    - Optimal rank: 16 (limited resources + moderate complexity)

Existing Approaches Fail:

- Uniform rank 32: Hospital A wastes capacity, Hospital C can barely train
- Resource-based: Hospital A gets rank 64, Hospital C gets rank 5, but C's data is actually more complex than its resources suggest

Our Solution: Domain-aware allocation gives Hospital A rank 64, Hospital B rank 32, Hospital C rank 16 based on both domain complexity AND resources.

Technical Approach

Component 1: Domain Complexity Analysis

1.1 Data Distribution Analysis

For each client, extract features from the frozen pre-trained model:

```python
def analyze_data_distribution(client_data, pretrained_model):
    """
    Analyze local data distribution to understand domain complexity
    """

    # Extract embeddings using frozen pre-trained model
    embeddings = []
    for batch in client_data:
        embedding = pretrained_model.get_embedding(batch)
        embeddings.append(embedding)

    embeddings = np.array(embeddings)

    # Compute statistics
    statistics = {
        'mean': np.mean(embeddings, axis=0),
        'std': np.std(embeddings, axis=0),
        'min': np.min(embeddings, axis=0),
        'max': np.max(embeddings, axis=0),
        'n_samples': len(embeddings)
    }

    return statistics, embeddings
```

1.2 Diversity Metrics

Measure how diverse the data is within the domain:

```python
def compute_diversity_metrics(embeddings):
    """
    Compute multiple diversity metrics

    High diversity → need higher rank
    Low diversity → can use lower rank
    """

    # Metric 1: Entropy-based diversity
    kmeans = KMeans(n_clusters=10, random_state=42)
    clusters = kmeans.fit_predict(embeddings)
    cluster_counts = np.bincount(clusters)
    entropy = -np.sum((cluster_counts / len(embeddings)) *
                      np.log(cluster_counts / len(embeddings) + 1e-10))

    # Metric 2: Variance-based diversity
    distances = pairwise_distances(embeddings)
    avg_distance = np.mean(distances)

    # Metric 3: Intrinsic dimensionality
    intrinsic_dim = estimate_intrinsic_dimension(embeddings)

    return {
        'entropy': entropy,
        'avg_distance': avg_distance,
        'intrinsic_dimension': intrinsic_dim
    }
```

1.3 Task Difficulty Estimation

Estimate how difficult the task is for this domain:

```python
def estimate_task_difficulty(client_data, pretrained_model, task_type):
    """
    Estimate task difficulty based on:
    1. How well pre-trained model performs (baseline)
    2. Task-specific characteristics
    3. Domain-specific factors
    """

    # Get baseline performance
    baseline_accuracy = evaluate_pretrained_model(client_data,
pretrained_model)

    # Task difficulty inversely related to baseline performance
    task_difficulty = 1.0 - baseline_accuracy

    # Adjust based on task type
    if task_type == 'classification':
        n_classes = len(np.unique(client_data.labels))
        task_difficulty *= np.log(n_classes) / np.log(10)

    elif task_type == 'regression':
        target_variance = np.var(client_data.targets)
        task_difficulty *= np.log(target_variance + 1)

    # Normalize to [0, 1]
    task_difficulty = np.clip(task_difficulty, 0, 1)

    return task_difficulty
```

1.4 Domain Complexity Score

Combine all factors into a single complexity score:

```python
def compute_domain_complexity_score(
    diversity_metrics,
    task_difficulty,
    data_size,
    data_imbalance
):
    """
    Combine multiple factors into overall domain complexity score

    Score ∈ [0, 1]:
    - 0: Simple domain (low diversity, easy task, small dataset)
    - 1: Complex domain (high diversity, hard task, large dataset)
    """

    # Normalize individual metrics
    entropy_score = diversity_metrics['entropy'] / np.log(10)
    distance_score = normalize_distance(diversity_metrics['avg_distance'])
    dim_score = diversity_metrics['intrinsic_dimension'] / 100

    # Combine with weights
    complexity_score = (
        0.3 * entropy_score +
        0.2 * distance_score +
        0.2 * dim_score +
        0.2 * task_difficulty +
        0.1 * normalize_size(data_size)
    )

    # Adjust for data imbalance
    if data_imbalance > 0.8:
        complexity_score *= 1.2

    return np.clip(complexity_score, 0, 1)
```

Component 2: Resource Profiling

2.1 Computational Resource Assessment

```python
def profile_computational_resources(device):
    """
    Profile available computational resources on client device
    """

    # GPU resources
    gpu_memory_total = get_gpu_memory_total()
    gpu_count = get_gpu_count()
    gpu_compute_capability = get_gpu_compute_capability()

    # CPU resources
    cpu_cores = get_cpu_count()
    cpu_frequency = get_cpu_frequency()

    # Memory resources
    available_disk = get_available_disk()
    available_ram = get_available_ram()

    return {
        'gpu_memory_gb': gpu_memory_total,
        'gpu_count': gpu_count,
        'gpu_compute_capability': gpu_compute_capability,
        'cpu_cores': cpu_cores,
        'cpu_frequency': cpu_frequency,
        'available_disk_gb': available_disk,
        'available_ram_gb': available_ram
    }
```

2.2 Maximum Supportable Rank Calculation

```
def calculate_max_supportable_rank(
    resource_profile,
    model_config,
    batch_size=32,
    safety_margin=0.2
):
    """
    Calculate maximum LoRA rank that can be supported given resources
    """

    # GPU memory constraint
    gpu_memory_available = resource_profile['gpu_memory_gb'] * (1 -
safety_margin)

    # Estimate memory per rank
    n_lora_layers = model_config['n_lora_layers']
    hidden_dim = model_config['hidden_dim']

    memory_per_rank = (
        2 * n_lora_layers * hidden_dim * 4 / 1e9 +
        2 * n_lora_layers * hidden_dim * 4 / 1e9 +
        model_config['model_size_gb'] * 0.5
    )

    max_rank_memory = int(gpu_memory_available / memory_per_rank)

    # Training time constraint
    training_time_per_rank = estimate_training_time(
        resource_profile,
        rank=1,
        batch_size=batch_size
    )

    max_rank_time = int(max_training_time / training_time_per_rank)

    # Communication constraint
    max_rank_comm = estimate_max_rank_for_communication(
        resource_profile,
        communication_budget
    )

    # Take minimum of all constraints
    max_rank = min(max_rank_memory, max_rank_time, max_rank_comm)
```

```
    # Quantize to standard values
    standard_ranks = [8, 16, 32, 64, 128]
    max_rank = max([r for r in standard_ranks if r <= max_rank], default=8)

    return max_rank
```

Component 3: Domain-Aware Rank Allocation

3.1 Rank Allocation Algorithm

```
def allocate_domain_aware_lora_rank(
    complexity_score,
    max_supportable_rank,
    min_rank=8,
    base_rank=16
):
    """
    Allocate optimal LoRA rank based on domain complexity and resources
    """

    # Base rank
    rank = base_rank

    # Increase based on complexity
    complexity_multiplier = 1 + 3 * complexity_score
    rank = int(rank * complexity_multiplier)

    # Constrain by maximum supportable rank
    rank = min(rank, max_supportable_rank)

    # Ensure minimum rank
    rank = max(rank, min_rank)

    # Quantize to standard values
    standard_ranks = [8, 16, 32, 64, 128]
    rank = min(standard_ranks, key=lambda x: abs(x - rank))

    return rank
```

3.2 Allocation Pipeline

```
def allocate_lora_ranks_for_all_clients(
    clients_data,
    pretrained_model,
    model_config,
    task_type='classification'
):
    """
    Allocate LoRA ranks for all clients in federated setting
    """

    allocations = {}

    for client_id, client_data in clients_data.items():
        # Step 1: Analyze data distribution
        stats, embeddings = analyze_data_distribution(
            client_data,
            pretrained_model
        )

        # Step 2: Compute diversity metrics
        diversity = compute_diversity_metrics(embeddings)

        # Step 3: Estimate task difficulty
        difficulty = estimate_task_difficulty(
            client_data,
            pretrained_model,
            task_type
        )

        # Step 4: Compute domain complexity score
        complexity = compute_domain_complexity_score(
            diversity,
            difficulty,
            len(client_data),
            compute_data_imbalance(client_data)
        )

        # Step 5: Profile resources
        resources =
profile_computational_resources(client_device[client_id])

        # Step 6: Calculate max supportable rank
        max_rank = calculate_max_supportable_rank(
```

```
            resources,
            model_config
        )

        # Step 7: Allocate optimal rank
        allocated_rank = allocate_domain_aware_lora_rank(
            complexity,
            max_rank
        )

        allocations[client_id] = {
            'rank': allocated_rank,
            'complexity_score': complexity,
            'max_supportable_rank': max_rank,
            'diversity_metrics': diversity,
            'task_difficulty': difficulty,
            'resources': resources
        }

    return allocations
```

Experimental Validation

Datasets

Domain 1: Medical Imaging

- Cardiology: 10,000 cardiac images
- Oncology: 8,000 tumor images
- Neurology: 6,000 brain images

Domain 2: Finance

- Retail Banking: 50,000 transaction records
- Investment Banking: 30,000 market records
- Insurance: 20,000 claim records

Domain 3: Scientific Literature

- Physics: 5,000 papers
- Biology: 4,000 papers
- Computer Science: 6,000 papers

Experimental Setup

Baselines:

1. **Uniform Rank**: All clients use rank 32
2. **Resource-Based**: Rank proportional to GPU count
3. **Random Allocation**: Random rank per client
4. **Oracle**: Optimal rank determined by exhaustive search

Metrics

1. **Accuracy**: In-domain accuracy, cross-domain accuracy
2. **Efficiency**: Communication cost, computation time
3. **Fairness**: Accuracy variance across clients
4. **Allocation Quality**: How close to oracle allocation

Expected Results

- 8-12% accuracy improvement over uniform ranks
- 25-30% communication cost reduction
- Automatic allocation without manual tuning
- Allocation quality within 5-10% of oracle

Implementation Roadmap

Technology Stack

Core Libraries

- PyTorch 2.0+
- Transformers (Hugging Face)
- PEFT
- NumPy
- Scikit-learn

Code Structure

```
project-2-domain-aware/
├── README.md
├── setup.py
├── requirements.txt
├── src/
│   ├── allocation/
│   │   ├── data_analyzer.py
│   │   ├── complexity_metrics.py
│   │   ├── resource_profiler.py
│   │   └── rank_allocator.py
│   ├── utils/
│   │   ├── metrics.py
│   │   └── visualization.py
│   └── main.py
├── experiments/
└── tests/
```

Timeline and Milestones

Phase	Duration	Milestones
1. Literature Review	Weeks 1-2	Understand LoRA, federated learning, domain adaptation
2. Algorithm Design	Weeks 3-4	Design complexity metrics and allocation algorithm
3. Implementation	Weeks 5-8	Implement all components
4. Experimentation	Weeks 9-14	Run experiments on all domains
5. Analysis & Writing	Weeks 15-20	Analyze results, write paper
6. Revision & Submission	Weeks 21-24	Revise based on feedback, submit

---

Expected Outcomes

Research Contributions

1. Domain-aware LoRA allocation algorithm for heterogeneous federated learning
2. Domain complexity metric that predicts required model capacity
3. Comprehensive evaluation on multiple domains
4. Insights into relationship between domain characteristics and required LoRA rank

Expected Results

- 8-12% accuracy improvement over uniform ranks
- 25-30% communication cost reduction
- Automatic allocation without manual tuning

Conclusion

This project presents **Domain-Aware LoRA Allocation for Heterogeneous Federated Learning**. The work automatically determines optimal LoRA ranks based on domain complexity and computational resources, eliminating the need for manual tuning.

Key Innovation: Explicit modeling of domain complexity combined with resource constraints to optimize LoRA rank allocation.

Expected Impact: 8-12% accuracy improvement, 25-30% communication cost reduction, automatic allocation without manual tuning.

Timeline: 8-10 months from start to publication.